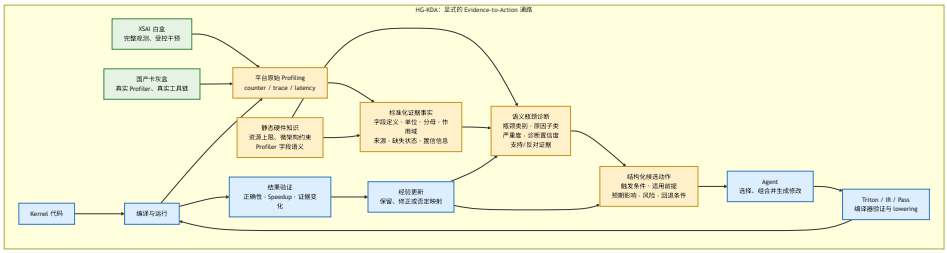
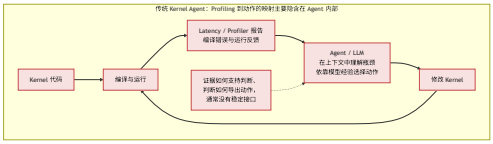
传统 Kernel Agent 与 HG-KDA：Profiling 到优化动作的区别

> 图的核心：传统 Kernel Agent 通常把 profiling 结果作为 Agent 的上下文；HG-KDA 把“标准化证据事实 → 语义瓶颈诊断 → 结构化候选动作”做成显式、可检查、可验证的系统接口。





两个中间层的输出示例

标准化证据层输出的是带有明确测量语义的事实，而不是瓶颈结论：

```yaml
metric: compute.vector_active
value: 0.20
unit: ratio
denominator: kernel_elapsed_cycles
scope: kernel
aggregation: mean
source:
  platform: vendor_A
  raw_metric: vec_pipe_active_cycles
availability: measured
```

语义瓶颈层把多条证据与硬件知识组合成诊断假设：

```yaml
bottleneck: compute_underutilization
subtype: insufficient_parallelism
severity: high
confidence: 0.78
supporting_evidence:
  - compute.vector_active = 0.20
  - execution.occupancy = 0.18
  - scheduler.eligible_work = low
contradicting_evidence:
  - memory.bandwidth_utilization = 0.35
alternative_hypothesis:
  - memory_latency_starvation
```

读图时需要区分的三个数值

1. **测量比例**：例如 `vector_active = 20%`。这是带明确分母的观测事实。
2. **瓶颈严重度**：例如 `severity = high` 或归一化分数 `0.8`。这是诊断系统定义的程度，不能解释为 80% 的运行时间。
3. **诊断置信度**：例如 `confidence = 0.78`。这是对诊断可靠性的校准概率或分数，也不是硬件利用率。

由于计算、访存和同步可以重叠，不同瓶颈的严重度通常不能相加为 100%。

一句话表述

传统流程：`Profiler` 输出 → `Agent` 内部推理 → 代码修改

HG-KDA：平台原始字段
→ 有来源和测量语义的标准化证据
→ 可检查的瓶颈诊断
→ 带适用条件和验证方法的候选动作
→ Agent/编译器执行
→ 用性能与证据变化验证诊断